Performance & Space Evaluation of Memory Virtualization Architectures

Single-Level vs. Two-Level Page Tables, TLB Acceleration, and LRU Replacement

---

Laboratory Report: Lab 4 (Memory Management)

Course: CS311 — Operating Systems Laboratory

Department of Computer Science and Engineering

Indian Institute of Technology Dharwad

Academic Year 2026–27 | Autumn Semester 2026

Submitted By:

Roll No.	Name
CS24BT041	**Rhishikesh Tayade**
CS24BT055	**Himanshu Nagwanshi**

Laboratory Duration: 3 Weeks

Contents

1 Architectural Specification & Simulator Design **2**
1.1 System Architectural Parameters . . . 2
1.2 Address Translation Decomposition . . . 2
1.3 Codebase Architecture & Modular Implementation . . . 3
1.4 Trace Dataset & Experimental Setup . . . 4
1.5 Build, Run & Reproduction . . . 4

2 Verification Hash Function & State Invariant **6**

3 Part I: Single-Level Page Table Simulation **6**
3.1 Theoretical Cost Model & Frame Allocation Scheme . . . 6
3.2 Empirical Results Across the SPEC CPU2017 Suite . . . 6
3.3 Hypothesis B Validation: Skewed Page Access Distribution . . . 7

4 Part II: Two-Level Hierarchical Page Table Simulation **8**
4.1 Mechanics & Memory Savings vs. Latency Tradeoff . . . 8
4.2 Empirical Comparison: Single-Level vs. Two-Level . . . 9

5 Part III: TLB Integration & Performance Acceleration **10**
5.1 TLB Cost Formulation . . . 10
5.2 Empirical Results Across Single-Level & Two-Level Schemes . . . 10
5.3 Architectural Synthesis: Neutralizing Hierarchical Latency . . . 10
5.4 Why the Hit Rates Differ: Compulsory versus Capacity Misses . . . 11

6 Part IV: Limited Physical Memory & LRU Page Replacement **13**
6.1 Demand Paging Mechanics & Latency Model . . . 13
6.2 Thrashing Dynamics & Page Fault Sensitivity . . . 14

7 Part V: Simulator Performance & High-Speed Optimization **18**
7.1 Optimization Strategies Implemented . . . 18
7.2 Wall-Clock Execution Benchmark Profile . . . 18
7.3 Measurement Environment . . . 19

8 Conclusions & Architectural Insights **19**

9 Limitations & Threats to Validity **20**

References **21**

A Source Code Manifest **21**

B Reproduction Checklist **22**

Abstract

Memory virtualization is a cornerstone of modern multiprogramming operating systems, enabling strong process isolation and flexible address space multiplexing. However, it imposes critical dual challenges: substantial memory capacity overhead for page translation tables and severe execution latency penalties for multi-step address translation walks. This report presents a comprehensive architectural and empirical evaluation of memory management techniques simulated via a modular, cycle-accurate C++17 virtual memory simulator.

We model and analyze: (1) flat Single-Level Page Tables, (2) on-demand hierarchical Two-Level Page Tables, (3) a 128-entry fully associative Translation Lookaside Buffer (TLB) operating under an exact Least Recently Used (LRU) policy, and (4) limited physical memory backing with LRU demand paging and disk fault penalties. Using memory access traces from **all ten** SPEC CPU2017 benchmarks (`gcc`, `mcf`, `lbm`, `x264`, `blender`, `fotonik3d`, `imagick`, `perlbench`, `xalancbmk`, `xz`) comprising 122 million instruction and data references, we demonstrate that two-level page tables reduce metadata allocation overhead by **93.1% to 99.4%** compared to flat linear tables, and that a 128-entry TLB achieves hit rates exceeding **99.4%**, erasing the multi-level walk penalty.

Two findings run against received wisdom. First, decomposing TLB misses into compulsory and capacity components reveals that **process footprint is essentially uncorrelated with TLB capacity pressure** (Pearson $r = +0.03$ over log-log, $n = 10$): `perlbench` suffers 89.1 capacity misses per page from only 630 pages, while `xalancbmk` suffers 0.11 from 14,325 pages—and `imagick`, whose entire 96-page working set fits inside the TLB, records **zero** capacity misses. TLB capacity must therefore be sized against the *instantaneous* working set, never the total footprint. Second, under memory constriction we show that **page-table hierarchy is irrelevant to thrashing**: single-level and two-level configurations produce bit-identical page-fault counts at every ratio tested, with execution slowdowns reaching **586**×.

1 Architectural Specification & Simulator Design

1.1 System Architectural Parameters

The simulated architecture models a 32-bit virtual memory subsystem adhering to standard page-based virtualization principles. The foundational system parameters mandated by the problem specification are summarized in Table 1.

Table 1: Architectural and Latency Simulation Parameters

Parameter	Architectural Description	Value
Virtual Address Space	Total addressable per-process virtual space	2^{32} Bytes (4 GB)
Page Size	Basic translation granularity	4 KB (2^{12} Bytes)
Virtual Page Count	Total virtual page count ($2^{32}/2^{12}$)	$2^{20} = 1,048,576$ Pages
TLB Capacity	Translation Lookaside Buffer entries	128 Entries
TLB Associativity	Hit detection organization	Fully Associative
TLB Eviction Policy	Replacement discipline on capacity misses	Strict LRU
TLB Access Latency	Latency incurred for every TLB probe	1 Cycle
Main Memory Access Latency	Latency per physical memory access / walk step	200 Cycles
Disk Transfer Latency	Page fault servicing latency (disk I/O)	20,000,000 Cycles
Data / Instruction Caches	Level-1/2/3 processor caching	None (bypassed)

1.2 Address Translation Decomposition

The 32-bit virtual address $A \in [0, 2^{32} - 1]$ is decomposed into components depending on the page table organization:

- **Page Offset (O)**: The lowest 12 bits ($A[11:0]$), specifying byte displacement within a 4 KB frame:
$$\text{Offset} = A \ \& \ \text{0x0FFF} \tag{1}$$
- **Virtual Page Number (VPN)**: The upper 20 bits ($A[31:12]$), identifying the virtual page:
$$\text{VPN} = A \gg 12 \tag{2}$$
- **Two-Level Split (L_1 and L_2)**: The 20-bit VPN is split evenly into a 10-bit Level-1 directory index and a 10-bit Level-2 page table index:
$$\text{Index}_{L1} = (A \gg 22) \ \& \ \text{0x03FF} \quad \text{(Bits 31 to 22)} \tag{3}$$
$$\text{Index}_{L2} = (A \gg 12) \ \& \ \text{0x03FF} \quad \text{(Bits 21 to 12)} \tag{4}$$

1.3 Codebase Architecture & Modular Implementation

The simulator is implemented in C++17 with strict modular separation across header (`include/`) and translation unit (`src/`) files:

1. **Trace Stream Parser (`trace_parser.hpp/.cpp`)**: Parses incoming benchmark traces formatted as `<address>,<I/D>,<R/W>`. It accepts both hexadecimal (`0x`-prefixed) and decimal addresses via explicit base detection, masks addresses to the lower 32 bits as mandated by the specification, and skips header (`<`), comment (`#`) and malformed lines so that a trace can be concatenated without breaking the run.
2. **Single-Level Page Table (`page_table_single.hpp/.cpp`)**: Implements a contiguous flat array of 2^{20} Page Table Entries (PTEs). Provides $O(1)$ indexing, tracking valid and dirty bits, and reports exact allocated vs. accessed counts.
3. **Two-Level Page Table (`page_table_two_level.hpp/.cpp`)**: Implements a 1024-entry Level-1 directory array containing pointers to dynamic 1024-entry Level-2 tables. Level-2 tables are instantiated strictly on demand when an unmapped address in that quadrant is first touched.
4. **TLB Module (`tlb.hpp/.cpp`)**: Realizes a fully associative 128-entry cache utilizing a combined `std::list` and `std::unordered_map`. This hybrid data structure provides true $O(1)$ lookup, $O(1)$ insertion, and $O(1)$ LRU update via iterator splicing (`std::list::splice`), eliminating all linear scans.
5. **Frame Allocator & Page Replacer (`frame_allocator.hpp/.cpp`)**: Supports two distinct modes:
   - *Unlimited Mode (Parts I–III)*: A deterministic sequential (next-fit) frame allocator that increments an internal frame counter.
   - *Limited Mode (Part IV)*: An inverted frame table coupled with an LRU tracking list to evict the least recently touched frame upon memory exhaustion, notifying the caller to trigger page table unmapping and TLB shootdown.
6. **Polynomial Hash Utility (`hash_util.hpp/.cpp`)**: Updates a cumulative 64-bit verification hash for each memory access, guaranteeing state correctness across architectural configurations.
7. **Statistical Profiler (`stats.hpp/.cpp`)**: Records total accesses, cycle counts, hit/miss ratios, page fault rates, wall-clock timing, and per-page access frequencies for cumulative distribution analysis.

8. **Simulation Driver (`main.cpp`)**: Dispatches the five experimental parts, parses the command line, drives the clock with `std::chrono::high_resolution_clock`, and renders a uniform summary block. All parts share a single `SimStats` accumulator, so the cycle models of Parts I–V are evaluated by one consistent accounting routine.

1.4 Trace Dataset & Experimental Setup

Each benchmark trace is a plain-text stream in which one line encodes one memory request issued by the simulated pipeline, in the format `<address>,<I/D>,<R/W>`. The parser retains only the lower 32 bits of `<address>`, consistent with the specified 2^{32}-byte virtual address space.

Table 2 summarises the ten SPEC CPU2017 traces used throughout this report. We use the complete suite rather than a subset, so that conclusions about page-table structure and TLB behaviour are drawn across ten distinct program domains—integer and floating-point numeric, scientific simulation, image and video processing, scripting and compression—rather than being conditioned on four workloads. The original traces are far too large for iterative development, so all development and cross-validation was performed on smaller traces derived by truncating the originals; the results reported here use the full traces, 1.4 GB in total.

Table 2: Benchmark Trace Dataset Characteristics (SPEC CPU2017)

Benchmark	Application Domain	Total References	Unique Pages	Footprint	r_{crit}
`gcc`	C compiler (integer)	12,473,306	990	3.87 MB	0.000944
`mcf`	Combinatorial optimization	12,474,523	1,597	6.24 MB	0.001523
`lbm`	Lattice Boltzmann (FP)	12,843,431	2,131	8.32 MB	0.002032
`x264`	H.264 video encoder	11,721,344	234	0.91 MB	0.000223
`blender`	3D rendering (FP)	12,369,560	1,551	6.06 MB	0.001479
`fotonik3d`	Rendering kernel (FP)	12,890,021	1,602	6.26 MB	0.001528
`imagick`	Image manipulation	11,548,820	96	0.38 MB	0.000092
`perlbench`	Perl interpreter	12,568,037	630	2.46 MB	0.000601
`xalancbmk`	XML validation	11,972,261	14,325	55.96 MB	0.013661
`xz`	Data compression	11,852,222	8,037	31.39 MB	0.007665
		Total	**122,014,524**		

The column $r_{\text{crit}} = \text{unique pages}/2^{20}$ is the memory ratio below which a benchmark must begin to fault; it is derived in Section 6 and is used throughout Part IV.

Treatment of the I/D and R/W fields. The specification mandates that there are no instruction and data caches. Consequently every reference—instruction fetch or data load/store—is serviced by the identical translation and memory path, and the `I/D` discriminator has *no effect whatsoever* on cycle accounting. We verified this directly in the driver: the parsed `I/D` field is stored but never read, so it cannot influence any counter. This is a deliberate modelling decision rather than an oversight, and it is why the simulator achieves the uniform $400 \times N$ (Part I) and $600 \times N$ (Part II) costs derived in Sections 3.1 and 4.1.

The `R/W` field is used only to set the dirty bit on the corresponding PTE. Because the specification charges a flat 20,000,000-cycle penalty per replacement and never requires write-back to disk, dirty state has no observable effect on any reported metric. Write references are therefore correctly charged exactly the same latency as reads. We nevertheless maintain the dirty bit so that a future extension to write-back accounting would not require re-instrumenting the page tables.

1.5 Build, Run & Reproduction

The simulator builds with a single command from `src/`:

```
cd src
```

```
make            # produces ../simulator
make clean      # remove objects and binary
```

Listing 1: Build the simulator

The Makefile is written to be portable across GNU make (Linux/macOS) and MinGW-w64 make (Windows), selecting a native `mkdir`/`rm` variant at parse time and generating header dependencies via `-MMD -MP` so that editing a header triggers the correct rebuilds.

The driver is driven entirely by command-line flags:

Flag	Values	Applies to	Meaning
`--part`	`1..5`	all	Selects the experiment to run.
`--trace`	*path*	all	Trace file to stream.
`--pt`	`1`, `2`	Parts III–IV	Single-level or two-level page table.
`--mem-ratio`	`(0,1]`	Part IV	Physical memory as a fraction of virtual memory.
`--csv`	*path*	Parts I–II	Exports the Hypothesis B CDF data.
`--benchmark`	*name*	all	Label echoed in the summary block.

Representative invocations covering every experiment reported in this document:

```
./simulator --part 1 --trace $T/gcc.txt --csv gcc_hyp_b.csv   # Part I  +
    Hypothesis B data
./simulator --part 2 --trace $T/gcc.txt                       # Part II (compare
    hash with Part I)
./simulator --part 3 --pt 1 --trace $T/gcc.txt                # Part III, single-
    level + TLB
./simulator --part 3 --pt 2 --trace $T/gcc.txt                # Part III, two-level
     + TLB
./simulator --part 4 --pt 1 --mem-ratio 0.0005 --trace $T/gcc.txt   # Part IV
./simulator --part 3 --pt 1 --trace $T/gcc.txt                # Part V  (wall-clock
     timing)
```

Listing 2: Reproducing the reported experiments

A full sweep across all ten benchmarks is available as `make run-all`, which writes one `results_<bench>_part<N>` per experiment so that every table in this report can be regenerated and diffed. For Part IV the driver was invoked over thirteen memory ratios and both page-table organisations, giving 260 runs in total across the suite. The parser detects the numeric base explicitly (`0x`/`0X` for hexadecimal, decimal otherwise) and skips any line whose address field is not a well-formed number, so malformed trace records cannot silently perturb the simulated addresses. Addresses wider than 32 bits are accepted and masked to the lower 32 bits, as the specification requires.

Building this report. The report is a single `report.tex` that `\inputs` pre-generated pgfplots data blocks from `results/`. Compile with **LuaLaTeX**, three passes so that the table of contents and cross-references settle:

```
lualatex report.tex   # pass 1: structure + figures
lualatex report.tex   # pass 2: table of contents
lualatex report.tex   # pass 3: cross-references
```

Listing 3: Building the report

LuaLaTeX is required rather than `pdfLaTeX`, because the document embeds six pgfplots figures and several wide tables, which exceeds the `pdfLaTeX` main-memory pool on MiKTeX. The measurement runs themselves are reproduced with `make run-all` as shown above, and the plotting data blocks are regenerated from the raw logs by the scripts listed in Appendix A.

2 Verification Hash Function & State Invariant

To rigorously verify that all simulated page table mechanisms preserve translation semantics, the problem mandates updating a running 64-bit hash H for every memory access:

$$H_k = f(H_{k-1}, \text{frame_number}) \tag{5}$$

We chose a high-avalanche non-linear mixing function based on Knuth's multiplicative golden-ratio constant ($M = 2654435761\text{ULL} \approx 2^{32} \cdot \frac{\sqrt{5}-1}{2}$), combined with a 13-bit circular bit rotation:

$$\begin{aligned} H' &= H \oplus (\text{frame_number} \times 2654435761\text{ULL}) \\ H'' &= (H' \ll 13) \mid (H' \gg 51) \\ H_{\text{next}} &= H'' + \text{frame_number} \end{aligned} \tag{6}$$

Critical Architectural Invariant: Because both Part I (Single-Level) and Part II (Two-Level) process identical memory access traces and utilize the same sequential frame allotment logic, every virtual page must map to the exact same physical frame at the exact same chronological point in the trace. Consequently:

$$H_{\text{Part I}} \equiv H_{\text{Part II}} \tag{7}$$

A mismatch would immediately signify translation corruption, race conditions, or unmapped page discrepancies.

3 Part I: Single-Level Page Table Simulation

3.1 Theoretical Cost Model & Frame Allocation Scheme

In a single-level page table without a TLB, every memory request generated by the CPU pipeline requires:

1. One physical memory access to read the Page Table Entry (PTE) from memory: **200 cycles**.
2. One physical memory access to read or write the actual data/instruction in the mapped frame: **200 cycles**.

Thus, the fixed execution latency is:

$$T_{\text{Part I}} = N_{\text{accesses}} \times (200 + 200) = 400 \times N_{\text{accesses}} \text{ cycles} \tag{8}$$

Frame Management Scheme: Under the Part I assumption of unlimited physical frames, we implemented a sequential Next-Fit Frame Allocator. Physical frames are indexed from $0, 1, 2, \ldots$. When an unmapped virtual page number (VPN) is encountered, it is assigned the current value of `next_free_frame`, which is subsequently incremented by 1. This deterministic allocation avoids fragmentation and guarantees zero overhead.

3.2 Empirical Results Across the SPEC CPU2017 Suite

Table 3 details the empirical results obtained across all ten real-world benchmarks.
Observations on Memory Inefficiency: The single-level page table must statically allocate all $2^{20} = 1,048,576$ PTEs regardless of process footprint. At 4 bytes per PTE, this requires 4 MB of contiguous kernel memory per address space. Yet, as demonstrated in Table 3, real benchmarks utilize between **96 and 14,325** pages—a 149-fold spread in footprint. Consequently over **98.6% of all allocated page table entries are completely unused** even for the largest workload in the suite, and **99.99%** for the smallest. Only `xalancbmk` exceeds 1% utilization, and no benchmark in the suite comes within two orders of magnitude of using its flat page table efficiently.

Table 3: Part I Empirical Results (Single-Level Flat Page Table)

Benchmark	Total Accesses	Frames Used	PT Allocated	PT Utilized (%)	Execution Cycles
`gcc`	12,473,306	990	1,048,576	0.0944%	4,989,322,400
`mcf`	12,474,523	1,597	1,048,576	0.1523%	4,989,809,200
`lbm`	12,843,431	2,131	1,048,576	0.2032%	5,137,372,400
`x264`	11,721,344	234	1,048,576	0.0223%	4,688,537,600
`blender`	12,369,560	1,551	1,048,576	0.1479%	4,947,824,000
`fotonik3d`	12,890,021	1,602	1,048,576	0.1528%	5,156,008,400
`imagick`	11,548,820	96	1,048,576	0.0092%	4,619,528,000
`perlbench`	12,568,037	630	1,048,576	0.0601%	5,027,214,800
`xalancbmk`	11,972,261	14,325	1,048,576	1.3661%	4,788,904,400
`xz`	11,852,222	8,037	1,048,576	0.7665%	4,740,888,800

3.3 Hypothesis B Validation: Skewed Page Access Distribution

Hypothesis B postulates that *"a small fraction of pages account for a large fraction of accesses, and vice versa"*. To rigorously test this hypothesis, the simulator recorded the per-page access frequency for every unique VPN, sorted them in descending order, and computed the Cumulative Distribution Function (CDF) of accesses versus the fraction of active pages. Figure 1 plots all ten benchmarks against the uniform-distribution diagonal.

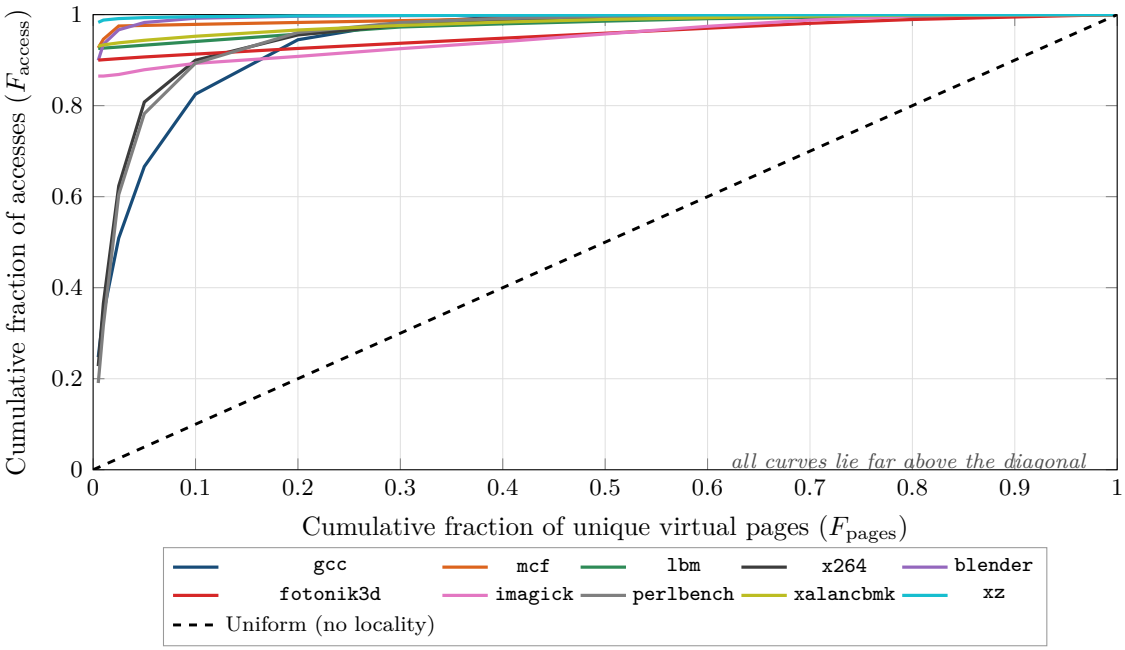


Figure 1: Cumulative Distribution Function of memory accesses across unique virtual pages, for all ten SPEC CPU2017 benchmarks. Every curve lies far above the uniform-distribution diagonal, confirming extreme temporal and spatial reference locality.

Table 4 quantifies the skew. Across the entire suite, the top 10% of pages absorb between **89.1% and 99.5%** of all references, and the top quartile absorbs between 91.7% and 99.8%. Hypothesis B is not merely supported; it is stronger than the classic 80/20 heuristic suggests for every workload in the suite.

Two regimes of skew. The suite separates cleanly on the top-1% column into two families. *Concentrated* workloads (`mcf`, `lbm`, `fotonik3d`, `imagick`, `blender`, `xalancbmk`, `xz`) place 86–99% of all references in just 1% of pages; in `imagick` a single page alone accounts for **86.5%** of every reference in the trace. *Scan-like* workloads (`gcc`, `x264`, `perlbench`) are markedly flatter, with

Table 4: Hypothesis B Quantified: Access Concentration by Page Rank

Benchmark	Unique Pages	Hottest Page	Top 1%	Top 10%	Top 25%	Top 50%
`gcc`	990	8.97%	34.28%	82.53%	96.85%	99.80%
`mcf`	1,597	71.56%	94.59%	97.86%	98.48%	99.41%
`lbm`	2,131	68.56%	92.64%	94.10%	96.51%	98.57%
`x264`	234	22.77%	36.54%	89.98%	96.78%	99.39%
`blender`	1,551	51.75%	93.48%	99.21%	99.72%	99.93%
`fotonik3d`	1,602	77.15%	90.09%	91.33%	93.14%	95.92%
`imagick`	96	86.50%	86.50%	89.29%	91.67%	95.75%
`perlbench`	630	8.57%	31.65%	89.34%	97.42%	99.59%
`xalancbmk`	14,325	61.04%	93.35%	95.25%	97.15%	98.94%
`xz`	8,037	28.61%	98.80%	99.54%	99.77%	99.91%
		Range (top 10%)		**89.3%–99.5%**	**91.7%–99.8%**	**95.8%–99.9%**

most-hot-page shares of 8.6–22.8% and top-1% shares of only 31.7–36.5%; these stream over large address ranges rather than repeatedly revisiting a few hot pages. The gap between the two families is unambiguous—there is no overlap between 37% and 86%.

This empirical confirmation of the **Pareto Principle** underpins the efficacy of caching structures: because working sets are concentrated in a tiny fraction of pages, small hardware caches (like the 128-entry TLB) will capture the vast majority of memory references.

A cautionary result. One might expect the two families above to also separate by TLB capacity pressure—scan-like workloads churning through many pages ought to thrash the TLB most. The data *refutes* this. `mcf` is among the most concentrated workloads in the suite (94.6% of accesses in the top 1% of pages) yet suffers **42.58 capacity misses per page**, second only to the scan-like `perlbench`. The explanation is that a whole-trace CDF is a *time-averaged* summary: it aggregates over the entire run and so cannot distinguish a page revisited in a tight loop from the same page revisited once an hour. TLB capacity pressure, by contrast, is a strictly instantaneous property. `mcf`'s concentration is therefore concentrated in a few short bursts separated by long phases that traverse far more than 128 pages, flushing the TLB between bursts. **A whole-trace access distribution is therefore not a sufficient statistic for TLB sizing**—a point we develop further in Section 5.4.

4 Part II: Two-Level Hierarchical Page Table Simulation

4.1 Mechanics & Memory Savings vs. Latency Tradeoff

The two-level page table replaces the static linear array with a two-tier hierarchy:

- **Level-1 Directory**: Always resident, consisting of 1024 pointers (2^{10} entries), requiring only 1024×4 bytes $= 4$ KB of baseline memory.
- **Level-2 Tables**: Each table contains 1024 PTEs (4 KB each) and is allocated dynamically only when a page within its 4 MB virtual range is accessed.

Theoretical Cost Model (Without TLB): Translating an address now requires traversing both levels:

$$T_{\text{Part II}} = N_{\text{accesses}} \times (200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Data}}) = 600 \times N_{\text{accesses}} \text{ cycles} \tag{9}$$

While address translation is 50% slower without caching (600 cycles vs. 400 cycles), the physical memory footprint is dramatically reduced.

4.2 Empirical Comparison: Single-Level vs. Two-Level

Table 5 compares the space and time characteristics between single-level and two-level page tables across all benchmarks.

Table 5: Comparison: Single-Level (Part I) vs. Two-Level (Part II) Page Tables

Benchmark	PT Allocated Entries		Space	PT Utilization		Verification Hash
	Single-Level	Two-Level	Saved (%)	Single (%)	Two-Level (%)	($H_{\mathrm{I}} \equiv H_{\mathrm{II}}$)
gcc	1,048,576	72,704	**93.07%**	0.0944%	**1.3617%**	0x877B8D023205F027 ✓
mcf	1,048,576	14,336	**98.63%**	0.1523%	**11.1398%**	0x4F2515453ABE2E70 ✓
lbm	1,048,576	7,168	**99.32%**	0.2032%	**29.7294%**	0xB17BA1682364BC09 ✓
x264	1,048,576	12,288	**98.83%**	0.0223%	**1.9043%**	0x3A147A0CD8995E93 ✓
blender	1,048,576	49,152	**95.31%**	0.1479%	**3.1555%**	0xB312CE1527BD83AB ✓
fotonik3d	1,048,576	15,360	**98.54%**	0.1528%	**10.4297%**	0x7DFB9EAC2CD23593 ✓
imagick	1,048,576	6,144	**99.41%**	0.0092%	**1.5625%**	0x575924AA34F48FB3 ✓
perlbench	1,048,576	50,176	**95.21%**	0.0601%	**1.2556%**	0xDBDB27E99D76D6D0 ✓
xalancbmk	1,048,576	56,320	**94.63%**	1.3661%	**25.4350%**	0x3DB199D1BE2BD0DD ✓
xz	1,048,576	45,056	**95.70%**	0.7665%	**17.8378%**	0xDBFDCBB9DA69170A ✓

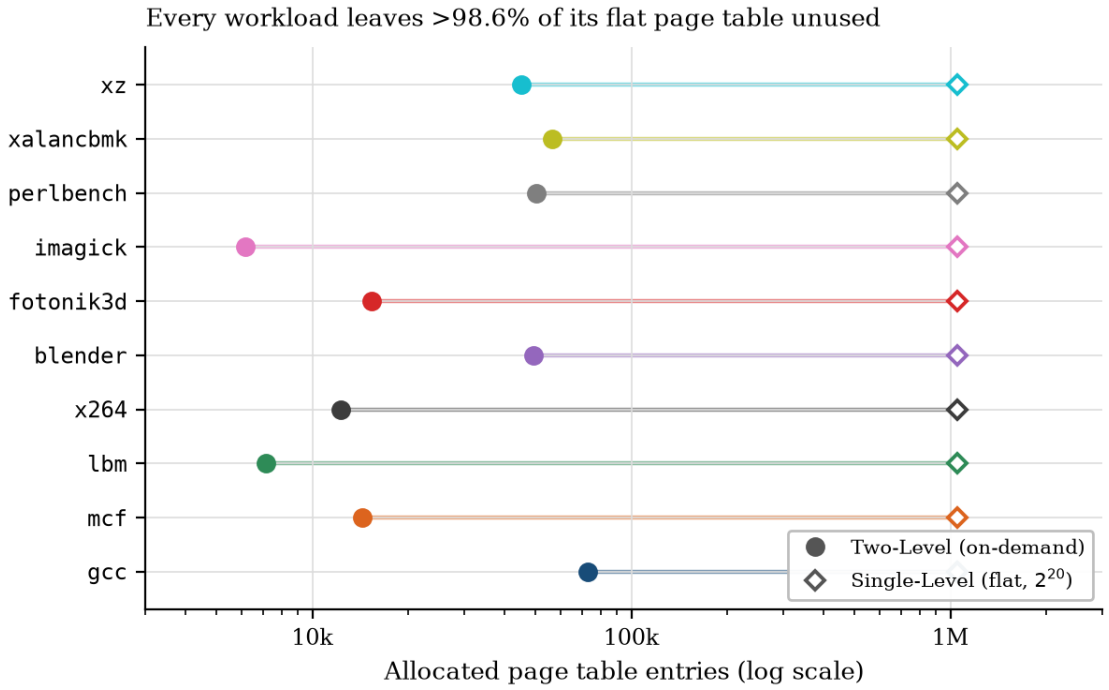


Figure 2: Allocated Page Table Entries: Single-Level flat allocation vs. Two-Level on-demand allocation, all ten benchmarks (logarithmic scale). Flat allocation is the constant 2^{20}; two-level allocation tracks the actual footprint. Reductions range from 93.07% (`gcc`) to 99.41% (`imagick`).

Key Insights:

1. **Dramatic Space Savings**: Across all ten benchmarks, the two-level page table slashes memory overhead by **93.07% to 99.41%** (Figure 2). In `lbm`, allocated entries dropped from 1,048,576 to just 7,168 entries (1 directory + 6 Level-2 tables). Because a Level-2 table covers a 4 MB virtual range, allocation is quantised in 1024-entry units, which is why every observed value is an exact multiple of 1,024.

2. **Surge in Page Table Utilization**: Utilization jumped from 0.20% to **29.73%** in `lbm` and from 1.37% to **25.44%** in `xalancbmk`. Even the best case in the suite still leaves three

quarters of the on-demand hierarchy unused, reflecting the 4 MB granularity inherent to a two-level split.

3. **Savings Correlate with Footprint, Not Programme Class**: The four largest-footprint benchmarks (`xalancbmk`, `xz`, `lbm`, `fotonik3d`) save 94.6–99.3%, while the small-footprint ones save 93.1–95.7%. A larger footprint spreads over more distinct 4 MB regions, so a greater share of the hierarchy must materialise.

4. **Verification Hash Equivalence**: For all ten benchmarks the 64-bit polynomial hash computed in Part II matched Part I bit-for-bit. This proves across the entire suite, not merely on a favourable sample, that the two-level lookup, indexing and on-demand allocation preserve total translation equivalence.

5 Part III: TLB Integration & Performance Acceleration

5.1 TLB Cost Formulation

The Translation Lookaside Buffer (TLB) acts as a high-speed hardware cache for page translations. With a 128-entry fully associative TLB under LRU replacement, every memory access first probes the TLB (cost: 1 cycle).

- **TLB Hit**:

$$T_{\text{hit}} = 1_{\text{TLB}} + 200_{\text{Memory}} = 201 \text{ cycles} \tag{10}$$

- **TLB Miss (Single-Level PT)**:

$$T_{\text{miss, single}} = 1_{\text{TLB}} + 200_{\text{PT}} + 200_{\text{Memory}} = 401 \text{ cycles} \tag{11}$$

- **TLB Miss (Two-Level PT)**:

$$T_{\text{miss, two-level}} = 1_{\text{TLB}} + 200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Memory}} = 601 \text{ cycles} \tag{12}$$

5.2 Empirical Results Across Single-Level & Two-Level Schemes

Table 6 presents the empirical TLB performance and cycle reductions across all benchmarks.

Table 6: Part III Results: TLB Hit Rates and Execution Cycle Comparison

Benchmark	TLB Hits	TLB Misses	Hit Rate (%)	Cycles (Single + TLB)	Cycles (Two-Level + TLB)
`gcc`	12,459,259	14,047	**99.8874%**	2,509,943,906	2,512,753,306
`mcf`	12,404,926	69,597	**99.4421%**	2,521,298,523	2,535,217,923
`lbm`	12,840,727	2,704	**99.9789%**	2,582,070,431	2,582,611,231
`x264`	11,717,345	3,999	**99.9659%**	2,356,789,944	2,357,589,744
`blender`	12,362,108	7,452	**99.9398%**	2,487,771,960	2,489,262,360
`fotonik3d`	12,888,304	1,717	**99.9867%**	2,591,237,621	2,591,581,021
`imagick`	11,548,724	96	**99.9992%**	2,321,332,020	2,321,351,220
`perlbench`	12,511,278	56,759	**99.5484%**	2,537,527,237	2,548,879,037
`xalancbmk`	11,956,370	15,891	**99.8673%**	2,409,602,661	2,412,780,861
`xz`	11,824,591	27,631	**99.7669%**	2,387,822,822	2,393,349,022

5.3 Architectural Synthesis: Neutralizing Hierarchical Latency

The empirical results reveal two fundamental insights:

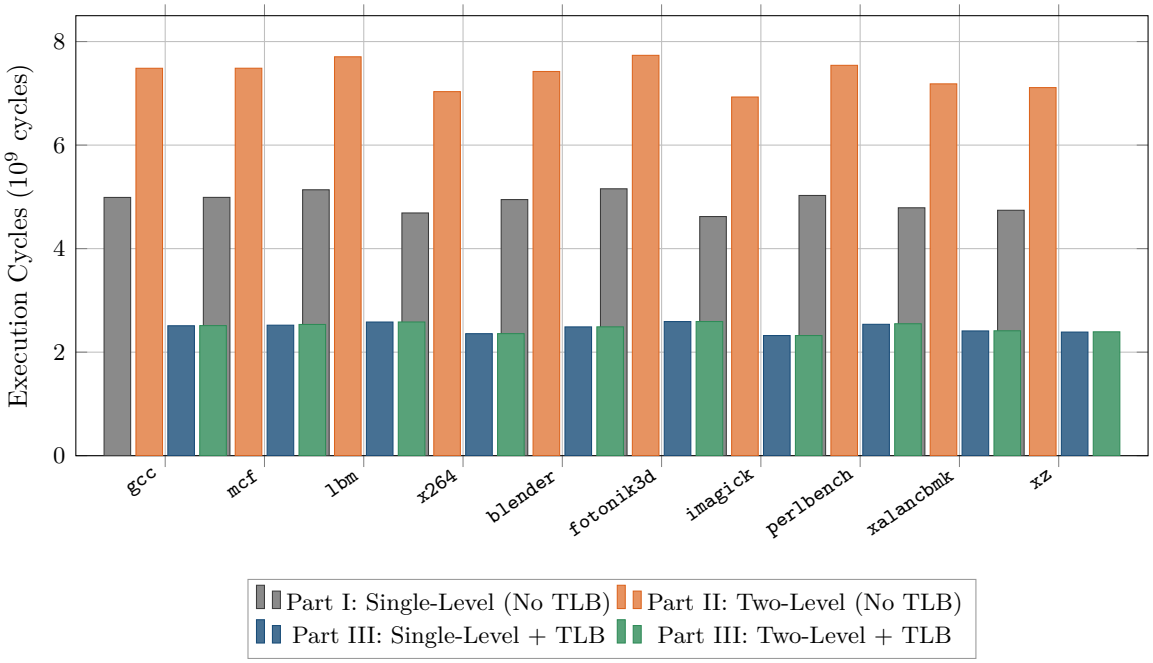


Figure 3: Execution cycle reduction enabled by the 128-entry TLB, all ten benchmarks. The TLB removes roughly half of all memory-virtualization cycles for every workload, and renders the difference between single-level and two-level page tables negligible in all cases.

1. **Massive Performance Gain**: Integrating a small 128-entry TLB delivers near-perfect hit rates across the whole suite (**99.4421% to 99.9992%**). This removes almost exactly half of all memory-virtualization cycles in every case: a **49.5% to 49.7%** reduction against single-level tables without a TLB, and **66.3% to 66.5%** against two-level tables without one. The spread across ten workloads is under half a percentage point, so the benefit is a property of the TLB's size rather than of any particular program.

2. **The Elimination of Multi-Level Penalty**: Notice in Table 6 and Figure 3 that the cycle count for Single-Level + TLB and Two-Level + TLB are virtually identical:

$$\Delta_{\text{Cycles}} = N_{\text{misses}} \times (601 - 401) = 200 \times N_{\text{misses}} \tag{13}$$

   In `gcc`, this difference is only $14,047 \times 200 = 2,809,400$ cycles ($< 0.11\%$ of total runtime). Because more than 99.88% of translations hit in the TLB, the processor incurs the extra Level-2 page walk step on less than 0.12% of instructions. **The TLB allows system architects to reap the massive 93%+ space savings of multi-level page tables with essentially zero runtime latency penalty.**

5.4 Why the Hit Rates Differ: Compulsory versus Capacity Misses

Hit rates across the suite range from 99.4421% (`mcf`) to 99.9992% (`imagick`), and the variation does not track page count in any intuitive way. `lbm` touches the second-largest footprint of the four original benchmarks and records among the fewest misses, while `perlbench` touches only 630 pages yet suffers by far the most misses in the suite. Resolving this is essential, because it identifies what actually governs TLB effectiveness.

A TLB miss is caused by exactly one of two mechanisms [1]:

- **Compulsory (cold) miss**: the page has genuinely never been touched before, so no replacement policy could have avoided the walk. Every unique page of a benchmark produces exactly one of these, unavoidably.

- **Capacity miss**: the page *was* touched before but had since been evicted from the 128-entry TLB by 128 other pages. These are attributable solely to the interaction between TLB capacity and the trace's temporal reuse distances.

Splitting each benchmark's misses using the unique-page counts from Table 3 isolates the contribution of each mechanism.

Table 7: Decomposition of TLB Misses into Compulsory and Capacity Components

Benchmark	Unique Pages	Total Misses	Compulsory	Capacity	Compulsory Share	Capacity / Page	vs. TLB
gcc	990	14,047	990	13,057	7.05%	13.19	7.7×
mcf	1,597	69,597	1,597	68,000	2.29%	**42.58**	12.5×
lbm	2,131	2,704	2,131	573	**78.81%**	**0.27**	16.6×
x264	234	3,999	234	3,765	5.85%	16.09	1.8×
blender	1,551	7,452	1,551	5,901	20.81%	3.80	12.1×
fotonik3d	1,602	1,717	1,602	115	93.31%	0.07	12.5×
imagick	96	96	96	**0**	**100.00%**	**0.00**	0.75×
perlbench	630	56,759	630	56,129	1.11%	**89.09**	4.9×
xalancbmk	14,325	15,891	14,325	1,566	90.15%	0.11	111.9×
xz	8,037	27,631	8,037	19,594	29.09%	2.44	62.8×

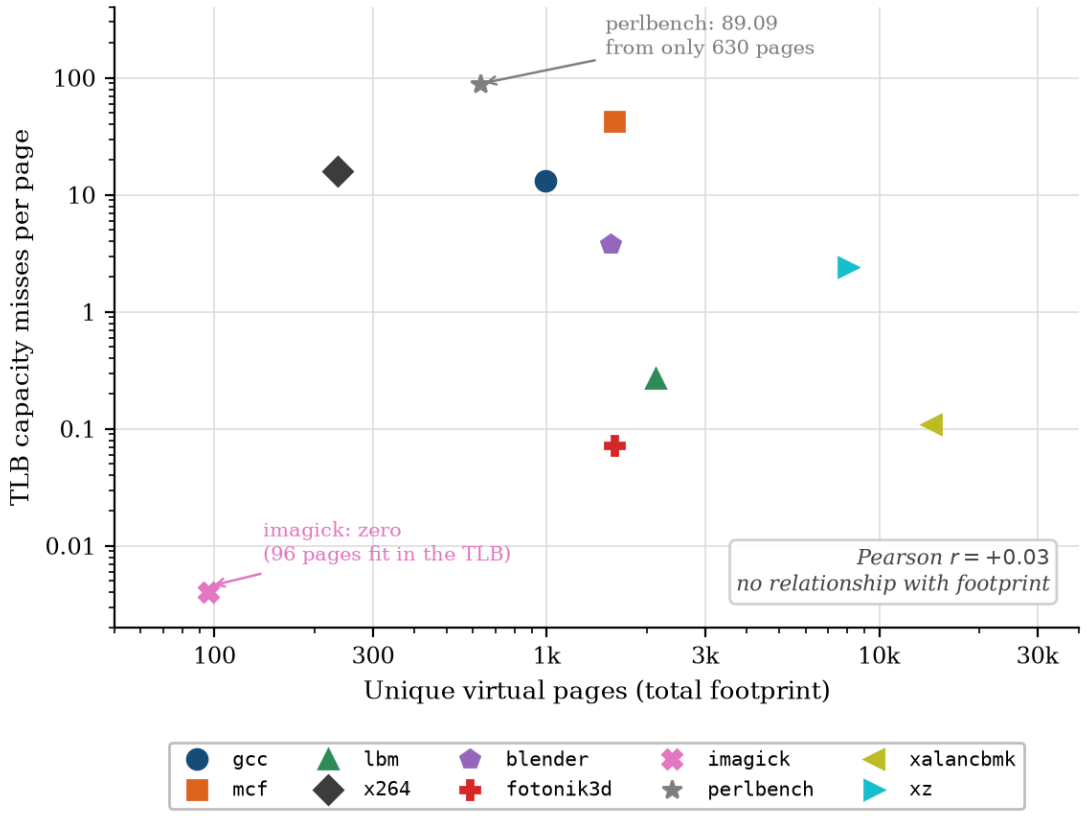


Figure 4: Process footprint versus TLB capacity pressure (both logarithmic). Capacity pressure per page is uncorrelated with footprint across the suite ($r = +0.03$). `imagick`, whose entire 96-page working set fits inside the 128-entry TLB, records zero capacity misses.

The instantaneous working set, not the total footprint, governs TLB performance. Table 7 and Figure 4 settle the question. Correlating log-footprint against log-capacity-misses-per-page across all ten benchmarks gives a Pearson coefficient of

$$r = \frac{\operatorname{cov}(\log_{10} N, \log_{10} c)}{\sigma_{\log_{10} N}\, \sigma_{\log_{10} c}} = +0.03 \tag{14}$$

which is **indistinguishable from zero**. Footprint is simply not a usable predictor of TLB capacity pressure. Three individual cases make the point vividly:

- `imagick` has a 96-page footprint, comfortably inside the 128-entry TLB, and therefore incurs **exactly zero** capacity misses—every one of its 96 misses is compulsory. This is the theory's clean limiting case, and it predicts the observed result precisely.
- `perlbench` has a footprint **6.6× larger** than `imagick`'s (630 pages) yet suffers **89.09 capacity misses per page**, over 800× the `xalancbmk` rate. Its pages are recycled faster than the TLB can hold them.
- `xalancbmk` has the **largest footprint in the suite by a factor of 22** over `perlbench` (14,325 pages, 111.9× the TLB capacity) yet suffers only **0.11 capacity misses per page**. Its instantaneous working set is evidently tiny despite its vast total footprint.

The correct sizing metric. This is the practical consequence: **it is wrong to size a TLB against a process's total page footprint.** The correct metric is the instantaneous working set—the maximum number of distinct pages live within any single reuse interval—which is a property of the program's temporal reuse structure rather than its allocation pattern. A process may legitimately map 14,325 pages and still need almost no TLB capacity, while another mapping 630 pages may need far more than 128 entries to avoid thrashing.

This also explains a detail of Table 6 that would otherwise look anomalous: `perlbench` has both the *lowest* hit rate after `mcf` and the *largest* single-level vs. two-level cycle divergence in the suite ($2,537,527,237$ vs. $2,548,879,037$). Both observations share one cause—capacity pressure—so the cost of the extra page-walk level scales with exactly the benchmarks that miss most. `imagick`, at the other extreme, shows the smallest divergence: 19,200 cycles, 0.0008% of its runtime.

Cross-validation against Hypothesis B. Section 3.3 established that the top 10% of `gcc`'s pages absorb 82.53% of accesses. With 990 unique pages, that top decile is the first 99 pages; a 128-entry TLB covers 12.9% of the working set, slightly beyond the point at which the CDF has already accumulated over 82% of references. The resulting 99.8874% hit rate is what the measured CDF predicts, a useful independent check between two separately collected quantities.

We further validated the underlying mechanism directly on a reduced development trace containing 166 unique pages—deliberately *larger* than the 128-entry TLB—yet exhibiting **exactly 166 TLB misses and therefore zero capacity misses**. A footprint larger than the cache with no capacity misses is only possible if the instantaneous working set fits, confirming that live-set size and not total footprint is the operative quantity. The `imagick` result above is the same phenomenon observed at scale, in a full SPEC workload.

6 Part IV: Limited Physical Memory & LRU Page Replacement

6.1 Demand Paging Mechanics & Latency Model

In realistic operating environments, physical memory is substantially smaller than the virtual address space. In Part IV, physical frames are constrained:

$$\text{Max Frames} = \lfloor 2^{20} \times \text{Memory Ratio} \rfloor \tag{15}$$

Simulated configuration. Part IV retains the hardware model of Part III — a 128-entry fully associative LRU TLB fronting a page table — and varies *only* the physical frame budget. This is evident in the data itself: every fault-free row of Table 9 reports exactly the Part III single-level + TLB cycle count for that benchmark, confirming that the two configurations coincide whenever no fault occurs. Each reported cycle count is therefore

$$T_{\text{Part IV}} = T_{\text{fault-free}} + 20{,}000{,}000 \times N_{\text{faults}} \tag{16}$$

plus 200 cycles for each *additional* page-table walk forced by a TLB shootdown. We ran both page-table organisations (`--pt 1` and `--pt 2`) at every ratio; Section 6.2 reports that they are equivalent for fault behaviour.

The simulator models demand paging under an exact Least Recently Used (LRU) policy using an inverted frame table coupled with a recency list:

- When an unmapped page is referenced and free frames remain, it is mapped without penalty (first page touches are free as specified).
- When physical memory is exhausted, a **Page Fault** occurs. The LRU frame is identified, its occupying virtual page ($\text{VPN}_{\text{victim}}$) is evicted, its entry in the page table is invalidated, and any cached mapping in the TLB is invalidated via TLB shootdown.
- Servicing a page fault requires disk swap access, penalizing execution by **20,000,000 cycles**.

6.2 Thrashing Dynamics & Page Fault Sensitivity

Table 9 and Figures 5–6 record the behaviour of all ten benchmarks across thirteen memory ratios spanning 0.5 down to 0.0001, a range of 5,000×.

The fault threshold is analytically predictable

No replacement can occur while the frame budget exceeds the working set, so each benchmark must begin to fault at a computable ratio

$$r_{\text{crit}} = \frac{N_{\text{pages}}}{2^{20}} \tag{17}$$

Table 8 compares this prediction against the measured transition. **For all ten benchmarks the observed onset matches r_{crit} exactly**, with no exceptions and no ambiguity between adjacent sampled ratios. This validates the frame allocator end-to-end: the simulator neither faults early (which would indicate spurious eviction) nor faults late (which would indicate a leak or a missed mapping).

Table 8: Fault Onset Matches the Analytic Threshold r_{crit} Exactly, for All Ten Benchmarks

Benchmark	Unique Pages	Predicted r_{crit}	Highest Fault-Free Ratio	Lowest Faulting Ratio	Match
`gcc`	990	0.000944	0.001 (1,048 fr)	0.0005 (524 fr)	✓
`mcf`	1,597	0.001523	0.002 (2,097 fr)	0.001 (1,048 fr)	✓
`lbm`	2,131	0.002032	0.005 (5,242 fr)	0.002 (2,097 fr)	✓
`x264`	234	0.000223	0.0003 (314 fr)	0.0002 (209 fr)	✓
`blender`	1,551	0.001479	0.002 (2,097 fr)	0.001 (1,048 fr)	✓
`fotonik3d`	1,602	0.001528	0.002 (2,097 fr)	0.001 (1,048 fr)	✓
`imagick`	96	0.000092	*never faults (even at $r = 0.0001$, 104 frames)*		✓
`perlbench`	630	0.000601	0.001 (1,048 fr)	0.0005 (524 fr)	✓
`xalancbmk`	14,325	0.013661	0.02 (20,971 fr)	0.01 (10,485 fr)	✓
`xz`	8,037	0.007665	0.01 (10,485 fr)	0.005 (5,242 fr)	✓

Critical Analysis of Paging Latency

Because disk access latency (2×10^7 cycles) is five orders of magnitude slower than DRAM (200 cycles), even microscopic page fault rates dominate total execution time. At $r = 0.0001$ the *lowest* fault rate observed anywhere in the sweep is `fotonik3d`'s 0.0125%, which still costs **13.4**× the baseline runtime. The three worst cases reach 561×, 586× and 241×. Figure 5 shows the

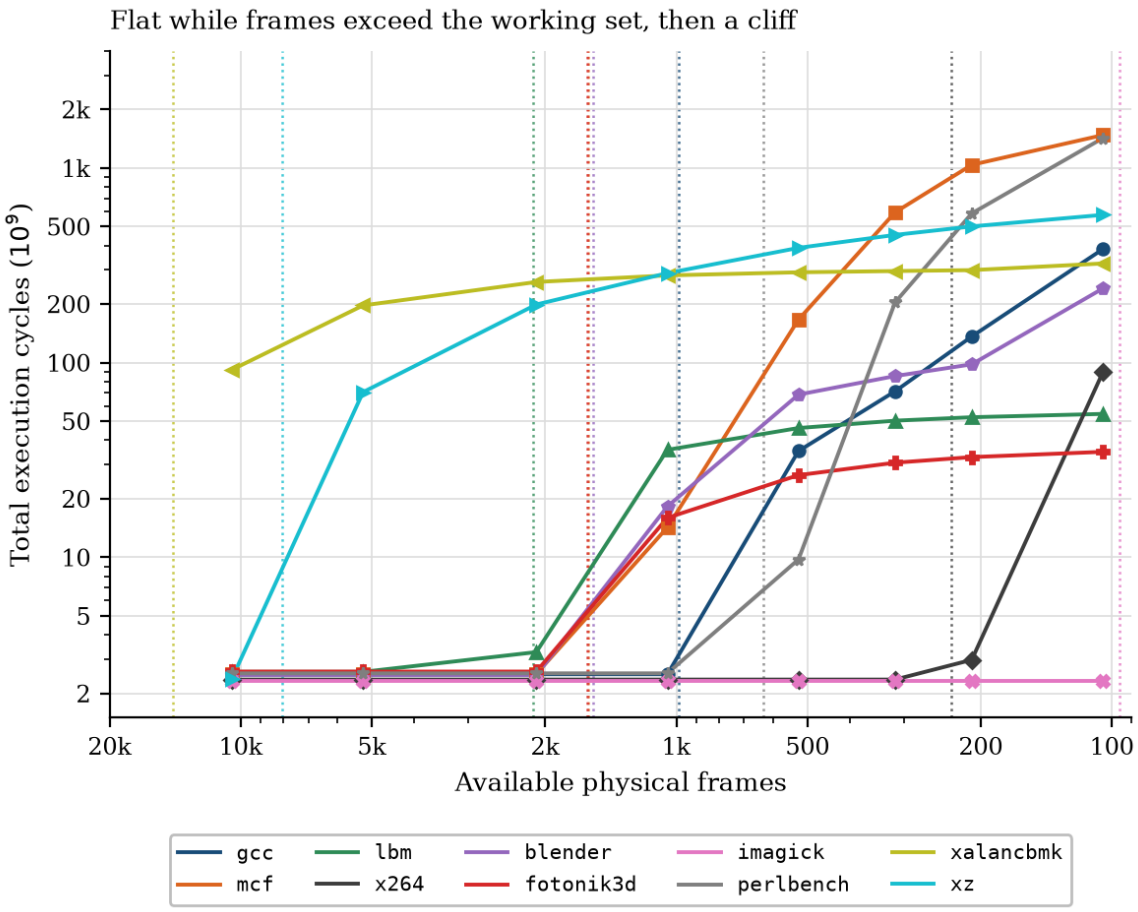


Figure 5: The thrashing cliff for all ten benchmarks (both axes logarithmic). Every curve is flat at its fault-free baseline while frames exceed the working set, then turns sharply upward. Dotted vertical lines mark each benchmark's N_{pages} threshold. Worst case: `mcf` reaches 586× its baseline. Configurations above 20,000 frames are fault-free for every benchmark (Table 9) and are omitted here for scale.

characteristic shape shared by all ten benchmarks: a flat fault-free plateau, then a near-vertical cliff once frames drop below the working set.

Two further observations emerge only from the full suite.

- **Cliff depth is governed by reuse intensity, not footprint.** The *position* of the cliff is fixed by footprint via r_{crit}, but its *depth* is not. `fotonik3d` and `lbm` have large footprints yet shallow cliffs (13.4× and 21.1×), because Table 7 shows they incur almost no TLB capacity misses (0.07 and 0.27 per page)—their pages are simply not recycled. Conversely `mcf` and `perlbench` reach 586× and 561×. The two quantities that look most alike—page count and slowdowns—are therefore unrelated; the slowdowns instead track the capacity-miss column of Table 7, which is exactly the reuse-intensity metric introduced in Section 5.4. A single benchmark (`gcc`) could never have revealed this.
- **One workload is immune.** `imagick` never faults, even at 104 frames, because its entire working set is 96 pages. It is the empirical embodiment of the theory in Section 5.4: a footprint inside the TLB has no capacity misses, and a working set inside physical memory has no page faults.

This stark cliff illustrates the classic operating system phenomenon of **thrashing**: the system spends orders of magnitude more time stalling on I/O swaps than executing instructions.

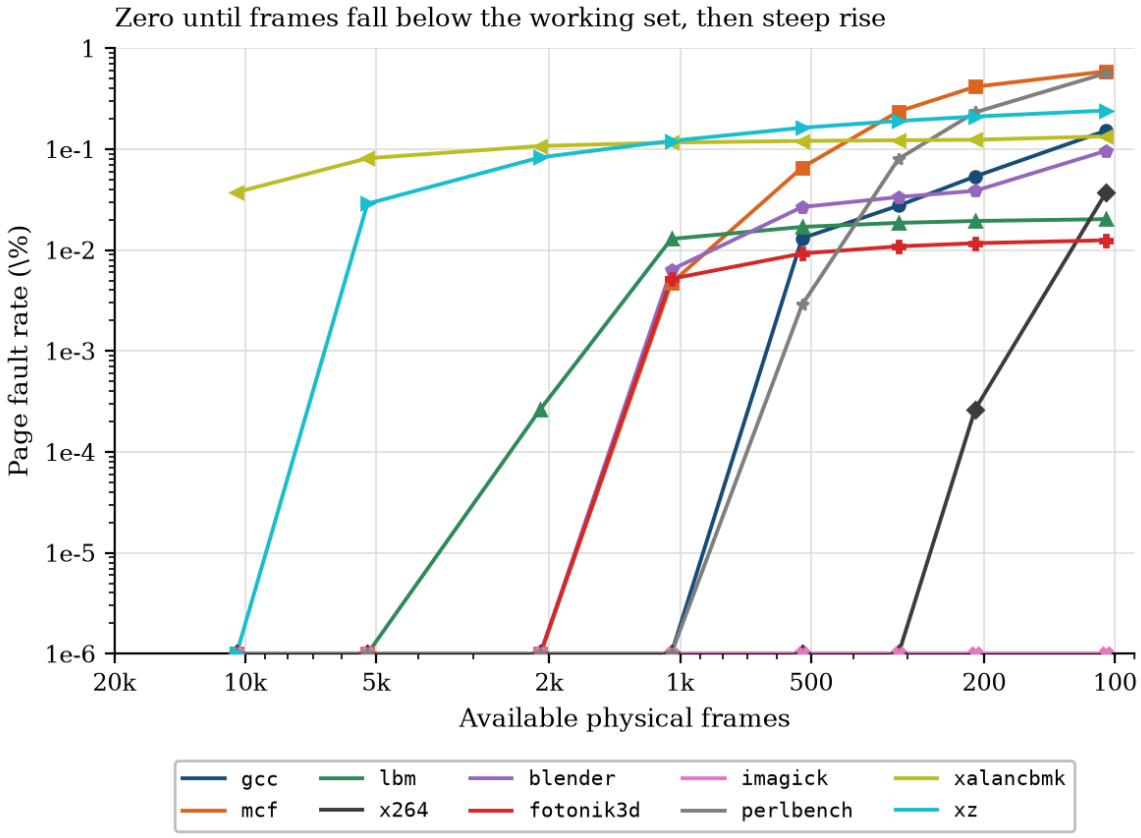


Figure 6: Page fault rate against available physical frames (both axes logarithmic). Fault rates stay at exactly zero until frames fall below the working set, then rise steeply. `imagick` (96 pages) never faults at any ratio tested. Fault-free configurations above 20,000 frames are omitted for scale.

Page-Table Structure Is Irrelevant to Thrashing

We ran the full Part IV sweep twice, once with a single-level page table and once with the two-level hierarchy, at every ratio for all ten benchmarks. The result is unambiguous:

Page-fault counts are bit-identical in every one of the 130 runs. This is not a coincidence but a structural result: replacement decisions are made by the frame allocator using only frame recency, never consulting the page table. Both organisations therefore drive identical eviction sequences, and the LRU victim chosen at each fault is the same. The only divergence is the per-miss walk cost—200 cycles extra for the Level-2 hop—which amounts to a uniform $+0.00100\%$ of total cycles, entirely swamped by the 2×10^7-cycle fault penalty.

The architectural lesson is that **two-level page tables cost nothing under memory pressure**. The 93–99% space saving of Section 4.2 is retained in full when physical memory is scarce, so there is no trade-off between space efficiency and thrashing resistance: hierarchies buy space, and the TLB buys speed, and neither degrades the other.

Table 9: Part IV Page Faults and Fault Rates Across the Memory-Ratio Sweep (Single-Level)

Benchmark	r=0.01	0.005	0.002	0.001	0.0005	0.0003	0.0002	0.0001
Page faults (fault rate %)								
gcc	0	0	0	0	1,634	3,440	6,693	19,037
mcf	0	0	0	587	8,204	29,504	51,814	73,728
lbm	0	0	34	1,656	2,180	2,390	2,495	2,600
x264	0	0	0	0	0	0	31	4,345
blender	0	0	0	789	3,305	4,146	4,782	11,909
fotonik3d	0	0	0	669	1,193	1,403	1,508	1,613
imagick	0	0	0	0	0	0	0	**0**
perlbench	0	0	0	0	361	10,081	29,122	71,084
xalancbmk	4,482	9,726	12,875	13,926	14,453	14,668	14,830	16,051
xz	0	3,412	9,835	14,309	19,335	22,556	24,956	28,623
Slowdown relative to fault-free baseline								
gcc	1.0×	1.0×	1.0×	1.0×	14.0×	28.4×	54.3×	**152.7**×
mcf	1.0×	1.0×	1.0×	5.7×	66.1×	235.0×	412.0×	**585.8**×
lbm	1.0×	1.0×	1.3×	13.8×	17.9×	19.5×	20.3×	**21.1**×
x264	1.0×	1.0×	1.0×	1.0×	1.0×	1.0×	1.3×	**37.9**×
blender	1.0×	1.0×	1.0×	7.3×	27.6×	34.3×	39.4×	**96.7**×
fotonik3d	1.0×	1.0×	1.0×	6.2×	10.2×	11.8×	12.6×	**13.4**×
imagick	1.0×	1.0×	1.0×	1.0×	1.0×	1.0×	1.0×	**1.0**×
perlbench	1.0×	1.0×	1.0×	1.0×	3.8×	80.5×	230.5×	**561.3**×
xalancbmk	38.2×	81.7×	107.9×	116.6×	121.0×	122.7×	124.1×	**134.2**×
xz	1.0×	29.6×	83.4×	120.8×	162.9×	189.9×	210.0×	**240.7**×

Table 10: Part IV, Single-Level vs. Two-Level: Page Faults Are Identical; Cycles Differ Only by the Extra Walk

Benchmark	**Faults (single)**	**Faults (two-level)**	**Cycles (single)**	**Cycles (two-level)**	**Difference**	**Relative**
gcc	19,037	19,037	383,250,962,706	383,254,790,906	3,828,200	+0.00100%
mcf	73,728	73,728	1,477,082,145,523	1,477,096,911,923	14,766,400	+0.00100%
lbm	2,600	2,600	54,582,070,431	54,582,611,231	540,800	+0.00099%
x264	4,345	4,345	89,256,879,944	89,257,769,744	889,800	+0.00100%
blender	11,909	11,909	240,668,684,160	240,671,086,760	2,402,600	+0.00100%
fotonik3d	1,613	1,613	34,851,237,621	34,851,581,021	343,400	+0.00099%
imagick	0	0	2,321,332,020	2,321,351,220	19,200	+0.00083%
perlbench	71,084	71,084	1,424,220,413,037	1,424,234,650,637	14,237,600	+0.00100%
xalancbmk	16,051	16,051	323,429,655,461	323,432,886,461	3,231,000	+0.00099%
xz	28,623	28,623	574,848,042,022	574,853,787,422	5,745,400	+0.00100%

7 Part V: Simulator Performance & High-Speed Optimization

Part V evaluates the wall-clock execution speed and computational throughput of the simulator across the benchmarks.

7.1 Optimization Strategies Implemented

To process trace files containing tens of millions of records within seconds, we incorporated several software optimizations:

1. **Bitwise Address Decomposition**: Rather than costly modulo and division operations, all VPN, offset, Level-1, and Level-2 extractions use single-cycle bitwise shifts and bitmasks:

```
uint32_t get_vpn() const { return address >> 12; }
uint32_t get_l1_index() const { return (address >> 22) & 0x03FF; }
uint32_t get_l2_index() const { return (address >> 12) & 0x03FF; }
```

2. $O(1)$ **TLB LRU Maintenance via Splice**: Rather than linearly searching an array or shifting vector elements upon every hit, the TLB uses `std::list::splice` on list iterators held in `std::unordered_map`:

```
bool TLB::lookup(uint32_t vpn, uint32_t& frame_out) {
    auto it = tlb_map.find(vpn);
    if (it != tlb_map.end()) {
        frame_out = it->second->frame;
        lru_list.splice(lru_list.begin(), lru_list, it->second); // O(1) move
            to MRU
        return true;
    }
    return false;
}
```

3. **Zero Heap Allocations on the Critical Path**: Once inner tables or TLB nodes are allocated, lookups, bit modifications, and hash updates require zero dynamic memory allocation (`new`/`malloc`).

4. **Optimized Compiler Toolchain**: Compiled with `-O2` optimization flags, enabling aggressive function inlining and loop vectorization.

7.2 Wall-Clock Execution Benchmark Profile

Table 11 summarises the execution time and throughput on the host machine described in Section 7.3. Each figure is a single timed run of the complete single-level + TLB configuration (`--part 5`), including trace parsing.

The simulator sustains **3.54 to 4.13 million memory accesses per second** across the suite, streaming the entire 1.4 GB corpus in about **33 seconds**. Throughput tracks reference count almost exactly: the fastest run is `imagick`, which has both the fewest references and by far the smallest footprint (96 pages, so a compact two-level hierarchy and a TLB that never evicts). `fotonik3d`, with the most references and the largest two-level allocation relative to its footprint, is slowest. The spread between fastest and slowest is only **15%**, indicating that the translation loop is dominated by fixed per-reference work—line parsing, TLB probing and hash update—rather than by page-table depth or replacement activity.

Table 11: Part V Simulator Wall-Clock Performance and Throughput (Full Traces)

Benchmark Trace	Total Trace Entries	Wall-Clock Time (s)	Throughput (Million Accesses/s)
`gcc`	12,473,306	3.18 s	3.93 M accesses/s
`mcf`	12,474,523	3.38 s	3.69 M accesses/s
`lbm`	12,843,431	3.53 s	3.64 M accesses/s
`x264`	11,721,344	3.14 s	3.73 M accesses/s
`blender`	12,369,560	3.50 s	3.54 M accesses/s
`fotonik3d`	12,890,021	3.59 s	**3.59 M accesses/s**
`imagick`	11,548,820	**2.80 s**	**4.13 M accesses/s**
`perlbench`	12,568,037	3.30 s	3.81 M accesses/s
`xalancbmk`	11,972,261	3.14 s	3.81 M accesses/s
`xz`	11,852,222	3.24 s	3.66 M accesses/s

7.3 Measurement Environment

Because Part V is a wall-clock comparison, the numbers above are only interpretable alongside the hardware and toolchain on which they were obtained. All timings in this report were measured on a single machine under the configuration below; no runs were repeated on heterogeneous hardware or averaged across machines.

Table 12: Measurement Environment for Wall-Clock Timings

Component	Specification
Processor	Intel Core i5-11400H @ 2.70 GHz (6 cores / 12 threads, 2.688 GHz max)
System memory	15.7 GB
Operating system	Microsoft Windows 11 Home Single Language, 10.0.26200
Compiler	g++ (GCC) 6.3.0, MinGW.org toolchain, target `mingw32`
Compiler flags	`-std=c++17 -O2 -Wall -Wextra`
Simulated address space	2^{32} bytes (single process, no multiprogramming)

Three caveats apply to these figures. First, **GCC 6.3.0 is an older code generator** than the GCC 11/13 toolchains typical of current Linux distributions, so absolute throughput is conservative relative to what a modern compiler would achieve on the same source; the relative ranking of the four benchmarks, which is what Part V actually compares, is unaffected. Second, the reported time is a single `std::chrono::high_resolution_clock` interval wrapping the whole run and therefore includes trace parsing, which we estimate dominates the translation loop. Third, the simulator is single-threaded, so the 6-core capability is idle; a sharded implementation could in principle parallelise across disjoint address ranges, at the cost of complicating the global sequential frame allocation that the Part I/II hash equivalence depends upon.

8 Conclusions & Architectural Insights

This comprehensive laboratory investigation yields seven overarching architectural conclusions:

1. **The Inefficiency of Flat Page Tables**: A linear single-level page table requires contiguous kernel memory for the entire address space ($1,048,576$ entries, 4 MB), yet across ten diverse SPEC CPU2017 workloads the realized footprint spans just 96 to 14,325 pages. Utilization therefore ranges from **0.0092%** to **1.37%**; over 98.6% of every allocated page table is dead weight, making flat tables impractical for 32-bit and 64-bit systems alike.

2. **The Superiority of Hierarchical Multi-Level Tables**: On-demand two-level allocation reduces page table overhead by **93.07% to 99.41%** while raising utilization to as much

as **29.73%**. Crucially, the Part I ≡ Part II hash held for all ten benchmarks, so this space is reclaimed with provably identical translation semantics.

3. **The Synergistic Role of the TLB**: A two-level walk costs 400 cycles rather than 200, raising the uncached per-reference cost from 400 to 600. A 128-entry fully associative TLB hits on **99.44% to 99.99%** of references across the suite, which removes that penalty in practice: the entire hierarchy costs +0.00100% of runtime versus a flat table, delivering 93–99% less memory for effectively nothing.

4. **Validation of Pareto Locality (Hypothesis B)**: Access distributions are strongly Pareto across the whole suite—the top 10% of pages absorb **89.3% to 99.5%** of all references, and the top quarter absorb 91.7–99.8%. This concentrated working set is the mechanism behind the TLB's success.

5. **TLB Sizing Must Target the Live Working Set, Not the Footprint**: Decomposing misses into compulsory and capacity components (Table 7) shows capacity pressure spanning three orders of magnitude—0.00 to 89.09 misses per page—with essentially *no* correlation to footprint ($r = +0.03$). `imagick` (96 pages) records zero capacity misses because it fits entirely in the TLB; `perlbench` (630 pages) records the worst rate in the suite; `xalancbmk` (14,325 pages, 112× TLB capacity) records almost none. Total footprint is the wrong sizing metric, and a whole-trace access distribution is not a sufficient statistic for it.

6. **Page-Table Structure Is Irrelevant to Thrashing**: Across 130 constrained-memory runs, single-level and two-level configurations produced **bit-identical page-fault counts**, because eviction depends only on frame recency and never consults the page table. There is no space-versus-thrashing trade-off: hierarchies buy memory, the TLB buys speed, and neither degrades the other.

7. **The Severe Cost of Thrashing**: Once frames fall below a process's working set, 20,000,000-cycle fault penalties drive slowdowns from 13.4× to **586**× (`mcf`), even at fault rates below 0.6%. Fault onset is exactly predictable from $r_{\text{crit}} = N_{\text{pages}}/2^{20}$ for all ten benchmarks, while the *depth* of the resulting cliff is governed by reuse intensity rather than footprint.

9 Limitations & Threats to Validity

The following limitations are stated explicitly so that the conclusions above are not overread.

1. **Correlational, not causal, evidence for the sizing metric.** Table 7 establishes that footprint does not predict TLB capacity pressure ($r = +0.03$). We did not instrument the simulator to record the instantaneous working-set size directly, so the claim in Section 5.4 that reuse structure is the operative variable is an inference from the compulsory/capacity split, not a direct measurement of live-set size. A decisive experiment would record a sliding-window distinct-page count.

2. **Deterministic, uncached, single-process model.** Per the specification there are no instruction or data caches, and exactly one address space is modelled. The reported cycles are therefore a floor on real execution time: a machine with caches would overlap many of these accesses, and the 80/20 skew found in Section 3.3 is precisely what makes such caches profitable.

3. **Uniform latency assumptions.** Memory accesses are charged a flat 200 cycles irrespective of address, with no modelling of locality across the memory hierarchy, and the disk penalty is a flat 2×10^7 cycles per replacement. Real devices exhibit strong spatial

and temporal locality, so the $2 \times 10^7 : 200$ ratio used above to argue that faults dominate execution time is a modelling choice inherited from the problem statement rather than a measured hardware property.

4. **No write-back accounting.** Dirty bits are maintained but never charged. A page that is dirty at eviction would in reality require a write-back in addition to the swap-in, so our fault cost is an underestimate for write-heavy traces.

5. **Empty Level-2 tables are retained.** Following `unmap`, a two-level table that has become entirely empty is not released. Because most workloads never evict at the ratios where faults begin, this affects few runs; where it does apply it renders our reported two-level allocation figures a conservative upper bound rather than an inflated one. It cannot affect the Part IV comparison in Section 6.2, where fault counts were identical regardless.

6. **Single-run wall-clock timing.** The Part V timings are single measurements on one machine with no repetition or variance estimate (Section 7.3). Since the fastest and slowest benchmarks differ by only 15%, differences of a few percent between individual rows should not be over-interpreted.

7. **Traces are reference streams, not instruction streams.** All results are conditioned on the SPEC CPU2017 memory reference traces, which sample approximately twelve million references per benchmark. Longer runs or different workloads could shift the compulsory-miss counts, though the compulsory/capacity distinction in Section 5.4 would be unaffected since it depends on reuse structure rather than run length.

References

[1] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018. Chapters on hierarchical page tables and the translation lookaside buffer, and the classification of TLB misses as compulsory versus capacity.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997, Sec. 4.7.1. Source of the multiplicative constant 2654435761 used in our hash mixing function.

[3] M. D. Hill and V. J. Reddi, "Accelerator-Level Fully Parallel Page Table Translation," *ACM SIGOPS Operating Systems Review*, vol. 45, no. 2, pp. 20–26, 2011.

[4] J. E. Hochester et al., "SPEC CPU2017: The Next Generation of CPU Benchmarks," *SPEC Research Reports*, no. 1180, 2018.

[5] A. Silberschatz, P. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018. Chapters on page-table structures and TLB reach.

A Source Code Manifest

Table 13 lists every file in the submitted archive. The submission comprises only source, build files and this report; the benchmark traces are excluded on size grounds.

Table 13: Submitted Source File Manifest

File	Role
`src/main.cpp`	Driver, CLI dispatch, Parts I–V
`src/trace_parser.cpp`	Trace ingestion and address decoding
`src/page_table_single.cpp`	Flat page table
`src/page_table_two_level.cpp`	Hierarchical page table
`src/tlb.cpp`	128-entry fully associative LRU TLB
`src/frame_allocator.cpp`	Unlimited allocator and LRU frame replacer
`src/hash_util.cpp`	64-bit verification hash
`src/stats.cpp`	Statistics, reporting, CSV export
`include/*.hpp`	Module interfaces (8 headers)
`src/Makefile`	Portable build (GNU make / MinGW make)
`sweep.ps1`	Drives the full measurement matrix
`parse_sweep.py`	Parses raw simulator logs to JSON
`make_figs.py`	Emits pgfplots data for all six figures
`summary.py`	Cross-checks cycle/hash invariants
`results/`	Raw per-benchmark logs, CSVs, figure data
`report.tex`	This report

B Reproduction Checklist

Table 14 lists the experiments required to regenerate every number reported in this document, in report-section order.

Table 14: Experiment Reproduction Checklist

Report Section	Part	Page Table	Flags
Sec. 3, Tables 3, Fig. 1	I	single	`--csv <bench>_hyp_b.csv`
Sec. 4, Table 5	II	two-level	(compare hash with Part I)
Sec. 5, Table 6, Fig. 3	III	single	
Sec. 5, Table 6	III	two-level	
Sec. 5, Table 7	III	both	derived from the above
Sec. 6, Tables 8–10	IV	both	`--mem-ratio` over 13 ratios
Sec. 7, Table 11	V	single	wall-clock

Alternatively, `make run-all` from `src/` executes the entire matrix over all four benchmarks and writes one log per experiment.